DEEP LEARNING 1 · CODING SEMINARS · WEEK 2

Research workflow in AI

Frameworks, GPU efficiency and good practices

All the code: github.com/daoterog/uva_dl1_seminars_2026

DEEP LEARNING 1 · CODING SEMINARS

The seven-week plan

W1	Snellius 101	SBUs, nodes, storage, first job
W2	**Research workflow in AI**	**Today**
W3	Implementing convolutions	Early CNNs to ConvNeXt
W4	Transformer inputs and outputs	Tensor shapes, ViTs in code
W5	Self-supervised learning	The state of the art
W6	Advanced generative learning	Flow matching, Drifting
W7	Exam prep	Past exams, together

LAST WEEK

Recap

How billing works, and one rule to remember.

RECAP · CPU BILLING

You are billed on the slice you request

PARTITION	ONE SLICE	CORES	MEMORY	COST
`rome`	1/8 of a node	16	28 GiB	16 SBU/h

`1 core`
Still reserves one slice



16
SBU / hour

`17 cores`
One core over a slice



32
SBU / hour

`1 core + 30 GiB`
Memory counts too



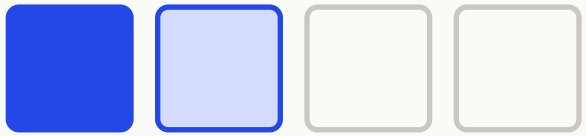
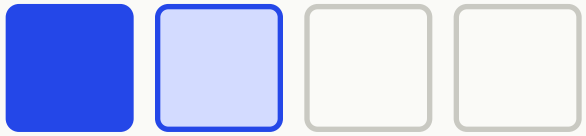
32
SBU / hour

You pay for the bigger of cores and memory, in whole slices.

RECAP · GPU BILLING

You are billed on the slice you request

PARTITION	ONE SLICE	GPU	CORES	MEMORY	COST
gpu_a100	1/4 of a node	1	18	120 GiB	128 SBU/h

`1 GPU + 18 cores` Fits in one slice		128 SBU / hour
`1 GPU + 32 cores` More cores than a slice		256 SBU / hour
`1 GPU + 130 GiB` More memory than a slice		256 SBU / hour

Over one slice, cores or memory: twice the price, same single GPU.

BEST WAY TO MANAGE YOUR SBUS

You are responsible for your SBU usage, not your agent.

Never run an agent in the cluster.

If an agent burns the whole course's credits, you pay for it. Not your agent.

TODAY

Research workflow

Why today's tools matter.

All of this is for your projects.

Snellius from last week. The material from today.

Both will be very handy when you tackle a project.

TODAY · WHY THIS MATTERS

Project courses in the Masters

Period 3	Fairness Accountability Confidentiality and Transparency in AI		
Period 5	Computer Vision 2	Deep Learning 2	Foundation Models
Period 6	Interpretability & Explainability in AI	Multimedia Analytics	Recommender Systems
Period 7	AI for Medical Imaging	Multilingual Natural Language Processing	
Period 8	Human-Centred Machine Learning	Information Retrieval 2	Vision for Autonomous Robots

TODAY · WHY THIS MATTERS

Research is a loop

Build
Write the code

`Lightning`

→

Run
Launch the experiment

`OmegaConf`
`GPU efficiency`

→

Measure
Get the right numbers

`torchmetrics`

→

Compare
See what changed

`wandb`

← **What you learn becomes the next experiment.**

Today: make every turn of the loop faster, and easier to trust.

PARTS 1 TO 4

Frameworks

What your training loop stops doing by hand.

THE BIGGER PICTURE

A guide, not the rule.

These are my picks. Other setups are just as valid.

But shared conventions make other people's code easy to read.

THE BIGGER PICTURE

Where you will meet them

Lightning



wandb



OmegaConf



via Hydra

Ai2

torchmetrics



every install

MosaicML

Not pivotal for uni. Useful for everything after it.

THE REPO FOR TODAY

One small, complete project.

```
git clone \
https://github.com/daoterog/uva_dl1_seminars_2026.git
```

PART 1 OF 4

Lightning

The loop you no longer write.

I have a dream.

That researchers' code is readable and organized.

No boilerplate.
One standard structure.

The Keras of PyTorch.

You write the science. Lightning handles the engineering.

Every AI project needs three things

1

The data

Load it, split it, batch it

2

The training loop

Epochs, devices, validation

3

What happens inside it

Your model, loss and optimiser

The second one is the same everywhere. The third one is your research.

PART 1 · LIGHTNING · THE BIGGEST WIN

Every project reads the same

WHERE TO LOOK

Want to see the data?

`DataModule`

Want to see the training?

`LightningModule`

THE WHOLE MAIN

```
dm = ExampleDataModule(cfg)
model = ExampleModule(cfg)
trainer = L.Trainer(...)

trainer.fit(model, datamodule=dm)
trainer.test(model, datamodule=dm)
```

No more hunting for the train function or guessing its arguments.

One piece for each thing

`LightningDataModule`

Your data, in one place

```
prepare_data
setup
train_dataloader
val_dataloader
```

Read the docs

`Trainer`

Runs the loop

```
.fit()
.validate()
.test()
.predict()
```

Read the docs

`LightningModule`

Inside the loop

```
training_step
validation_step
test_step
configure_optimizers
```

Read the docs

Every hook, and the order they run in: Hooks in Lightning

The DataModule skeleton

```
class ExampleDataModule(L.LightningDataModule):
    def prepare_data(self):
        """Download the data. Runs once per node."""

    def setup(self, stage):
        """Build the splits. Runs on every GPU."""

    def train_dataloader(self):
        """Return the training DataLoader."""

    def val_dataloader(self):
        """Return the validation DataLoader."""

    def test_dataloader(self):
        """Return the test DataLoader."""
```

The LightningModule skeleton

```
class ExampleModule(L.LightningModule):
    def __init__(self, cfg):
        """Build the model and its metrics."""

    def training_step(self, batch, batch_idx):
        """One training batch. Return the loss."""

    def validation_step(self, batch, batch_idx):
        """One validation batch. Log the metrics."""

    def test_step(self, batch, batch_idx):
        """One test batch. Log the metrics."""

    def configure_optimizers(self):
        """Return the optimiser and the LR schedule."""
```

The boring stuff? You no longer code it.

Devices. Mixed precision. Clipping. Checkpoints. Logging.

Not everything, but most of what you rewrite in every project.

The Trainer's main arguments

```
trainer = L.Trainer(
    max_epochs=30,                  # how long to train
    accelerator="auto",             # cpu, gpu
    devices="auto",                 # how many
    precision="bf16-mixed",         # mixed precision
    gradient_clip_val=1.0,          # gradient clipping
    accumulate_grad_batches=2,      # one step every 2 batches
    check_val_every_n_epoch=1,      # how often to validate
    log_every_n_steps=50,           # how often to log
    logger=WandbLogger(...),        # where the logs go
    callbacks=[...],                # checkpoints and your own hooks
    fast_dev_run=False,             # True: one batch, to catch bugs
    profiler=None,                  # "simple": where the time goes
)
```

Where it runs, and for how long

```
trainer = L.Trainer(
    max_epochs=30,                  # how long to train
    accelerator="auto",             # cpu, gpu
    devices="auto",                 # how many
    precision="bf16-mixed",         # mixed precision
    gradient_clip_val=1.0,          # gradient clipping
    accumulate_grad_batches=2,      # one step every 2 batches
    check_val_every_n_epoch=1,      # how often to validate
    log_every_n_steps=50,           # how often to log
    logger=WandbLogger(...),        # where the logs go
    callbacks=[...],                # checkpoints and your own hooks
    fast_dev_run=False,             # True: one batch, to catch bugs
    profiler=None,                  # "simple": where the time goes
)
```

Training tricks, one number each

```
trainer = L.Trainer(
    max_epochs=30,                  # how long to train
    accelerator="auto",             # cpu, gpu
    devices="auto",                 # how many
    precision="bf16-mixed",         # mixed precision
    gradient_clip_val=1.0,          # gradient clipping
    accumulate_grad_batches=2,      # one step every 2 batches
    check_val_every_n_epoch=1,      # how often to validate
    log_every_n_steps=50,           # how often to log
    logger=WandbLogger(...),        # where the logs go
    callbacks=[...],                # checkpoints and your own hooks
    fast_dev_run=False,             # True: one batch, to catch bugs
    profiler=None,                  # "simple": where the time goes
)
```

Seeing what your run is doing

```
trainer = L.Trainer(
    max_epochs=30,                  # how long to train
    accelerator="auto",             # cpu, gpu
    devices="auto",                 # how many
    precision="bf16-mixed",         # mixed precision
    gradient_clip_val=1.0,          # gradient clipping
    accumulate_grad_batches=2,      # one step every 2 batches
    check_val_every_n_epoch=1,      # how often to validate
    log_every_n_steps=50,           # how often to log
    logger=WandbLogger(...),        # where the logs go
    callbacks=[...],                # checkpoints and your own hooks
    fast_dev_run=False,             # True: one batch, to catch bugs
    profiler=None,                  # "simple": where the time goes
)
```

Catching bugs before you submit

```
trainer = L.Trainer(
    max_epochs=30,                  # how long to train
    accelerator="auto",             # cpu, gpu
    devices="auto",                 # how many
    precision="bf16-mixed",         # mixed precision
    gradient_clip_val=1.0,          # gradient clipping
    accumulate_grad_batches=2,      # one step every 2 batches
    check_val_every_n_epoch=1,      # how often to validate
    log_every_n_steps=50,           # how often to log
    logger=WandbLogger(...),        # where the logs go
    callbacks=[...],                # checkpoints and your own hooks
    fast_dev_run=False,             # True: one batch, to catch bugs
    profiler=None,                  # "simple": where the time goes
)
```

PART 1 · THE DOWNSIDE

The price is some flexibility

`Not every fit`	Some projects are not compatible, for specific reasons.
`Hard to bend`	Modifying or integrating Lightning is hard.
`But rare`	In my experience, these are very specific cases.

wandb

Runs you can compare.

Deep learning fails silently.

Often no crash, just a worse model.

You only catch it by watching the run.

Monitoring

To debug a run while it happens.

Experiment tracking

To document it, and to reproduce it later.

PART 2 · WHAT IS WANDB?

wandb keeps every run in one place

Track
Metrics and system stats, per run

Compare
Runs side by side

Reproduce
Commit and config stay with the run

Share
A link, not a screenshot

A guide, not the rule: MLflow and TensorBoard do this job too.

PART 2 · MONITORING

Every curve, for every run



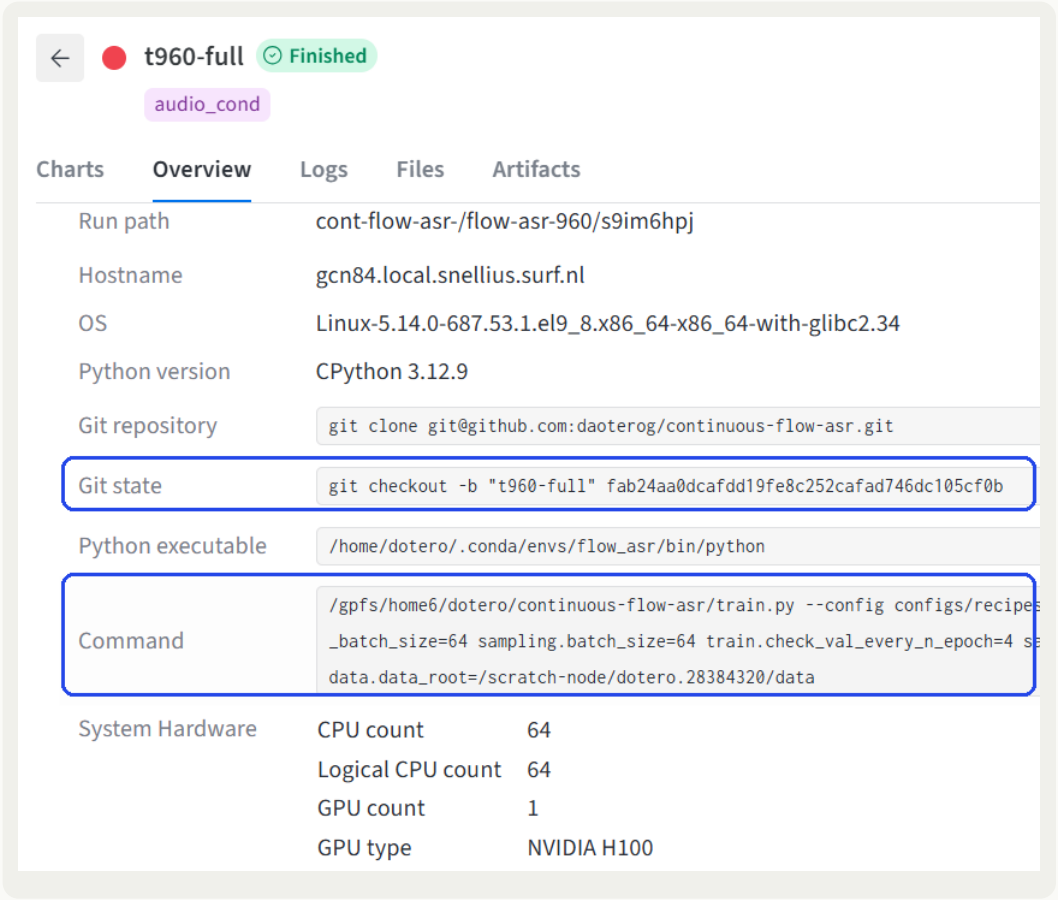
Spot the difference

NAME 3 of 6 listed	.NVA	DATA.COMPA	DATA.DATA_I	DATA.DIREC	DATA.ENV	DATA.EVAL_I
t960-ful...gma0.05		false	/scratch-node/d	true	snellius	64
t960-full		false	/scratch-node/d	true	snellius	64
t960-compact		true	/scratch-node/d	true	snellius	512

Every config key is a column.

What changed between two runs is one look away.

Everything you need to rerun it





The exact commit

The exact command

The full config

The WandbLogger's main arguments

```
logger = WandbLogger(
    entity="dl1-team",       # team or user that owns the runs
    project="cifar10",       # the folder your runs go into
    name="resnet-lr0.1",     # optional: if unset, wandb picks one
    group="lr-sweep",        # runs that belong together
    tags=["baseline"],       # labels to filter runs by
    config=config,           # every hyperparameter of the run
    offline=False,           # True: no internet, sync it later
)
trainer = L.Trainer(logger=logger)
```

OmegaConf

One config, in layers.

PART 3 · THE BIGGER PICTURE

It always starts with argparse

DAY 1

```
python train.py --lr 0.1 --epochs 30
```

THREE MONTHS LATER

```
python train.py --lr 0.2 --epochs 200 --batch-size 256 \
  --weight-decay 5e-4 --momentum 0.9 --nesterov \
  --label-smoothing 0.1 --num-workers 8 --precision bf16-mixed \
  --compile --eval-batch-size 1024 --seed 3 --group sweep-b \
  --model resnet18 --augment --drop-last --...
```

Which flags belong to which setting? One typo and you have run something else.

War story

Settings optimized for my laptop being run on Snellius.

Store the recipe, not the flags.

One file per setting.

Easier to read, harder to mess up, explicit about what you ran.

One project, many settings

WHERE IT RUNS

Local
Paths, few workers

Snellius
Scratch paths, more workers

One specific GPU
Batch size, precision

HOW BIG IT IS

Small experiment
Few epochs, frequent logging

Big run
Long, checkpoints, less logging

Each of these changes a lot of flags at once.

PART 3 · OMEGACONF

Four layers, each with one job

`configs/base.yaml`	What is trained	128
`configs/where/<machine>`	How to keep this machine busy	—
`--overlay <file.yaml>`	An experiment on top	512
`key=value`	The command line	1024

Follow batch_size: each layer overrides the one before, so 1024 wins.

The same example, in files

base.yaml

```yaml
data:
  batch_size: 128
  num_workers: 4
train:
  max_epochs: 10
  precision: 32-true
```

where/snellius_mig.yaml

```yaml
data:
  num_workers: 8
train:
  precision: bf16-mixed
```

experiments/large_batch.yaml

```yaml
data:
  batch_size: 512
optim:
  lr: 0.4
```

```
python train.py --where snellius_mig \
  --overlay configs/experiments/large_batch.yaml data.batch_size=1024
```

The calls you actually need

```
cfg = OmegaConf.load("base.yaml")              # read the YAML file
OmegaConf.set_struct(cfg, True)                # a typo is now an error

where = OmegaConf.load("snellius_mig.yaml")  # one layer on top
cfg = OmegaConf.merge(cfg, where)              # the later one wins

cli = OmegaConf.from_dotlist(args)             # key=value from the shell
cfg = OmegaConf.merge(cfg, cli)                # the command line goes last

cfg.data.batch_size                            # read it with dots
```

That is the whole idea. The real version is in src/config.py. Read the docs

torchmetrics

Metrics are state, not numbers.

Several GPUs, one correct metric.

In DDP, each GPU only sees its own part of the data.

torchmetrics syncs across devices for you.

One GPU or DDP

	`One GPU` The regular setting	`DDP` DistributedDataParallel
PROCESSES	One	**One per GPU**
DATA	All of it	**Each GPU has its own share**
USE IT	Your default	**Too slow or too big for one GPU**

From here on we are in DDP.

PART 4 · THE DDP SETTING

How the states are synced

	correct	total
GPU 0	1	37
GPU 1	5	63
Sum	**6**	**100**

```
compute()
6 / 100
6.0%
```

The sum is your dist_reduce_fx. It runs right before compute.

Accuracy on two GPUs, with DDP

GPU 0
1 correct out of 37

2.7%

GPU 1
5 correct out of 63

7.9%

Average the accuracies
Wrong

5.3 %

Add up the counts
6 correct out of 100

6.0%

So each GPU has to keep the counts, not the accuracy.

A state is what you keep, not the result.

For accuracy: correct and total.

States add up across batches and GPUs. Accuracies do not.

The Metric skeleton

```
class ExampleAccuracy(Metric):
    def __init__(self):
        super().__init__()
        self.add_state("correct", torch.tensor(0), dist_reduce_fx="sum")
        self.add_state("total", torch.tensor(0), dist_reduce_fx="sum")

    def update(self, logits, target):  # (B, C), (B,)
        preds = logits.argmax(dim=-1)
        self.correct += (preds == target).sum()
        self.total += target.numel()

    def compute(self):
        return self.correct / self.total
```

B: batch size. C: number of classes.

PART 4 · WITH LIGHTNING

Logging in Lightning

```
class ExampleModule(L.LightningModule):
    def __init__(self):
        super().__init__()
        self.val_acc = ExampleAccuracy()

    def training_step(self, batch, batch_idx):
        x, y = batch
        loss = ...
        self.log(
            "train/loss", loss, on_step=True, on_epoch=True,
            sync_dist=True, batch_size=len(y),
        )
        return loss

    def validation_step(self, batch, batch_idx):
        x, y = batch
        self.val_acc.update(self.model(x), y)
        self.log("val/acc", self.val_acc, on_step=False, on_epoch=True)
```

on_step
Every batch

on_epoch
Once, per epoch

sync_dist
Plain numbers, across GPUs

batch_size
Weights the epoch mean

KEEP THESE HANDY

Links

REPO	uva_dl1_seminars_2026
LIGHTNING	lightning.ai/docs/pytorch
WANDB	docs.wandb.ai
OMEGACONF	omegaconf.readthedocs.io
TORCHMETRICS	lightning.ai/docs/torchmetrics

COMING NEXT

Keeping the GPU busy

We will try to include this throughout the next weeks.